

HW1: A Bounded Agent for Bandit Alert Triage

COMS E6998 Section 019, Fall 2026 · Design of Production Agentic Systems

Instructor: Rahul Krishna · TA: In Keun Kim

Released Friday, September 25, 2026 · Due Friday, October 16, 2026, before 11:59 p.m.

1 Overview

You will build an agent that triages the alerts a static analyser raises on a Python repository. The handout fixes the interfaces: the command line, the configuration, the output file, the trace and the README. You design and build everything behind them. The rubric gives one point for each stage of the agent, so your grade shows which mechanisms exist and work.

The submission has two graded parts, a design report and an implementation, and you hand them in together. There is no separate design checkpoint. Instead, you may consult the instructors as often as you like, at any stage of the work.

Released	Friday, September 25, 2026
Due	Friday, October 16, 2026, before 11:59 p.m.
Weight	20% of the course grade, marked out of 20 (Section 9)
Format	Team assignment. One submission per team: design report and implementation together.
Late days	6 per student per semester, at most 3 on one assignment; each extends the deadline by 24 hours.

2 What you build

Your agent takes one pinned Python repository and works in two stages.

1. **Discovery.** Run Bandit 1.9.4 at **MEDIUM or higher severity** over the scan scope that the configuration sets (Section 4).
2. **Triage.** Label every alert **1** (potential vulnerability) or **0** (not a vulnerability), with a rationale and line evidence. It then writes `report.json`, with one row per alert, and `trace.jsonl`, the record of the run.

You write all of the code. The handout gives no starter code, stubs or tests. Any language and framework is allowed, but Bandit needs Python 3.12. The target code is read only: your agent reads it, and never installs or runs it.

3 Targets

Clone each target at its pinned commit. The root of that clone is `--input`. Your agent must run on all three targets.

Target	Repository	Commit	Role	Alerts
OWASP BenchmarkPython	https://github.com/OWASP-Benchmark/BenchmarkPython	f1291485808b66e20ddb6b01b10dc71b3df8c8ba	development	18 alerts
Radicale	https://github.com/Kozea/Radicale	eff8027f3dc4910be1659ee71f4b4a454ade5a8c	held out	10 alerts
SGLang	https://github.com/sgl-project/sglang	bbe9c7eeb520b0a67e92d133dfc137a3688dc7f2	held out	21 alerts

OWASP BenchmarkPython is the development set. Its answer key is open to you: the benchmark's own labels, in the file `expectedresults-0.1.csv` at the repository root. Read it to measure your agent and to refine its prompts, tools and stopping rules. **Your agent may not read it.** At run time the key is withheld from the agent's workspace (Section 7); an agent that reads it measures nothing.

Radicale and SGLang are held out. Their answer keys are hidden, and staff score these two targets after submission. They show whether your agent generalises or has only been tuned to the benchmark.

4 Required scan scope for each target

Required scan scope for each target

These three `[scan]` blocks are mandatory. Use them unchanged: graders run every submission with exactly these blocks. `paths` and `exclude` are globs relative to `--input`, and your Bandit wrapper expands them.

Radicale: 10 alerts

```
[scan]
paths = ["radicale"]
exclude = ["radicale/tests"]
min_severity = "MEDIUM"
bandit_version = "1.9.4"
```

SGLang: 21 alerts

```
[scan]
paths = ["python/sglang/multimodal_gen/runtime",
        "scripts/playground/replay_request_dump.py"]
exclude = []
min_severity = "MEDIUM"
bandit_version = "1.9.4"
```

OWASP BenchmarkPython: 18 alerts (test cases 00001 to 00099)

```
[scan]
paths = ["testcode/BenchmarkTest000*.py"]
exclude = []
min_severity = "MEDIUM"
bandit_version = "1.9.4"
```

The counts come from Bandit 1.9.4 on CPython 3.12, measured on fresh clones at the pinned commits. They count results of MEDIUM or higher severity, at any confidence.

5 Interface

The agent is a command-line program:

```
./their-agent --input <target checkout root> --output <out dir> --config config.toml
```

It writes two files into <out dir>: `report.json` (Section 6) and `trace.jsonl` (Section 8). You choose the name `their-agent`, and your README states it.

Your agent must read the configuration file. Staff ship `config.example.toml`, shown here with the Radicale values; for another target, replace its `[scan]` block with that target's block from Section 4.

config.example.toml

```
# HW1 agent configuration. Your agent must read this file:
# ./their-agent --input <target checkout root> --output <out dir> --config config.toml
#
# [scan] is fixed by staff for each target. Copy the block for your target from
# the brief's "Required scan scope for each target" section, unchanged. Graders
# replace it with the same staff block when they run your agent.
[scan]
paths = ["radicale"]           # globs relative to --input; your Bandit wrapper expands them
exclude = ["radicale/tests"]   # globs relative to --input
min_severity = "MEDIUM"       # required; do not lower
bandit_version = "1.9.4"       # required

# Your choice of provider and model. Read secrets from environment variables,
# never from this file.
[model]
provider = "anthropic"
name = "claude-sonnet-5"

[budget]
max_tokens = 200000            # the whole run, across all alerts
output_allowance = 1024        # max output tokens for each model call
max_steps = 200                # failsafe, separate from the token budget
```

The token budget covers the whole run, not each alert. When it runs out, every alert that is not yet classified still gets a row in `report.json`, with `status` set to `budget_exhausted`.

6 Output

`report.schema.json` (JSON Schema) is normative: your `report.json` must validate against it. The example below shows the format for a cutoff run over three alerts. It is a format illustration only; its labels are not answers.

report.example.json (format illustration; the labels are not answers)

```
{
  "target": {"input": "targets/radicale", "commit": "eff8027f3dc4910be1659ee71f4b4a454ade5a8c"},
  "scan": {"bandit_version": "1.9.4", "min_severity": "MEDIUM", "alert_count": 3},
  "budget": {"max_tokens": 20000, "used_tokens": 19480, "exhausted": true},
}
```

```

"alerts": [
  {
    "alert_id": "radicale-auth-oauth2-py-60-B113",
    "test_id": "B113",
    "path": "radicale/auth/oauth2.py",
    "line": 60,
    "severity": "MEDIUM",
    "confidence": "LOW",
    "label": 0,
    "status": "classified",
    "rationale": "Format illustration only: one or two sentences citing what the agent read.",
    "evidence": [{"path": "radicale/auth/oauth2.py", "start_line": 52, "end_line": 66}]
  },
  {
    "alert_id": "radicale-storage-multifilesystem-lock-py-102-B602",
    "test_id": "B602",
    "path": "radicale/storage/multifilesystem/lock.py",
    "line": 102,
    "severity": "HIGH",
    "confidence": "HIGH",
    "label": null,
    "status": "error",
    "rationale": "The model reply could not be parsed as a label.",
    "evidence": []
  },
  {
    "alert_id": "radicale-storage-multifilesystem-sync-py-79-B301",
    "test_id": "B301",
    "path": "radicale/storage/multifilesystem/sync.py",
    "line": 79,
    "severity": "MEDIUM",
    "confidence": "HIGH",
    "label": null,
    "status": "budget_exhausted",
    "rationale": "",
    "evidence": []
  }
]
}

```

Rules:

- There is exactly one row for each in-scope Bandit result. Rows are never dropped or merged, so `len(alerts)` equals `scan.alert_count`.
- Rows are ordered by `path`, then `line`, then `test_id`.
- `status` is one of `classified`, `budget_exhausted` or `error`.
- `label` is 1 or 0 when `status` is `classified`. Otherwise it is `null`.
- `used_tokens` is the sum of the usage that the provider reported.

7 Workspace and withholding

You build the workspace and the withholding yourselves; there is no deny list in the configuration. The reference pattern is cyberbird's `cyberbird/reactive/workspace.py`, in particular `resolve`, `AgentWorkspace` and `WITHHELD_GLOBS`: <https://github.com/comse6998-019/cyberbird/blob/cf7e96c8bd9e74470b99b7b754bb72392427cfc6/cyberbird/reactive/workspace.py>

Requirements:

1. The agent works in a disposable copy of `--input`. Its tools can reach only that copy.
2. Every tool path goes through one path check. The check refuses `..` paths, absolute paths, and symlinks that resolve outside the copy.
3. `.git/` is withheld on every target. The history of a full clone can hold commits made after the pinned one, including fixes.
4. On OWASP BenchmarkPython, the answer key `expectedresults-0.1.csv` (in general, `expectedresults*.csv`) is withheld from the agent. So is every `BenchmarkTest*` file except the case under investigation, because each case has a safe twin. You may still read the key yourselves (Section 3); only the agent's view is restricted.

8 Trace and trajectory

The trace is the agent's trajectory: the complete, ordered record of what one run did and who decided each step. Graders read it to check B3 to B8, so its format is fixed. You build it yourselves. The reference implementation is cyberbird's

cyberbird/plan_and_validation/trace.py (Trace, EventKind, TerminalStatus, Usage) and cyberbird/plan_and_validation/trajectory.py (trace to Mermaid sequence diagram):
https://github.com/comse6998-019/cyberbird/blob/cf7e96c8bd9e74470b99b7b754bb72392427cfc6/cyberbird/plan_and_validation/trace.py
https://github.com/comse6998-019/cyberbird/blob/cf7e96c8bd9e74470b99b7b754bb72392427cfc6/cyberbird/plan_and_validation/trajectory.py

Format:

- `<out dir>/trace.jsonl` holds one JSON object per line. It is append-only and flushed after every event, so that a crashed run leaves the trace that explains the crash.
- Every event has **step** (an integer from 1, increasing by 1), **ts** (Unix time, float), **run_id**, **alert_id** (the alert the event belongs to, or null for run-level events) and **kind**.
- **kind** is one of the closed set below. Any event may carry other fields as well.

kind	Written by	Required fields
model_call	the node that called the model	role , model , usage with input and output (integers, as the provider reported them; also cache_read and cache_write if the provider reports them; no summed total)
tool_request	the dispatcher	tool , args
tool_result	the dispatcher	tool , ok (boolean), and error when ok is false
routing	the edge function that chose the next node	decision , by (model or runtime)
state_change	any node, for updates not covered above	node ; when an alert finishes, also alert_status and label as written to report.json
terminal	the runtime, once, last	status (one of your terminal statuses, A6), usage_total

Invariants (README check R11, Section 10):

1. The file parses line by line, and every event has the required fields for its **kind**.
2. **step** starts at 1 and increases by exactly 1.
3. There is exactly one **terminal** event, and it is the last line, also for a run that crashed.
4. **usage_total** equals the sum of **usage** over all **model_call** events, and **input** + **output** summed over them equals **budget.used_tokens** in **report.json**.
5. For every alert in **report.json**, there is a **state_change** event whose **alert_status** and **label** equal that row's **status** and **label**.
6. Every **tool_request** is followed by exactly one **tool_result** for the same tool before the next **model_call**.

Trajectory command. Ship one command that takes a **trace.jsonl** file and an **alert_id**. Using the trace alone, it prints that alert's trajectory as a Mermaid **sequenceDiagram**. The model and the runtime are separate participants in the diagram. Its output for one real run is the sequence diagram required in C2.

9 Rubric

The assignment is marked out of 20, one point per item.

Part A: design report (7)

- | | |
|----|---|
| A1 | Behaviours. What the agent does for each alert, and what it must never do: run target code, write to the target, invent observations. |
| A2 | State machine. A diagram of the states and the transitions between them. |
| A3 | Nodes and edges. Each node, each edge marked as conditional or fixed, and who decides each transition (the model or the runtime). |
| A4 | Agent state. Each field with its reducer, its lifetime, who writes it, and whether the model sees it. |
| A5 | Tools. Contracts for at least 3 tools, with arguments, limits and refusal cases. |
| A6 | Start, stop and termination. A closed, mutually exclusive set of terminal statuses and what triggers each one. This includes the budget policy: what counts toward the budget, when it is checked, and what happens at the cutoff. |
| A7 | Workspace and withholding. Which files the agent must not see for each target and why, how they are withheld, and how they are restored. |

Part B: implementation (10)

- B1 **Bandit tool.** Reads `[scan]`, uses MEDIUM+ severity, uses Bandit 1.9.4, and parses the JSON output.
- B2 **Workspace, read and search tools.** Meets every requirement in Section 7.
- B3 **Dispatcher.** The runtime validates and runs every request. Unknown tools and bad arguments come back as refusal observations, not crashes. Only the runtime writes observations.
- B4 **Graph.** Implemented as the design in Part A describes.
- B5 **Tracer and trajectory.** Writes `trace.jsonl` in the Section 8 format with every invariant holding (R11): one event per step, flushed immediately, routing events that record who decided, and exactly one terminal event, even after a crash. Ships the trajectory command of Section 8.
- B6 **Budget capture.** Charges the usage the provider reports, not an estimate.
- B7 **Budget cutoff.** Checked before each model call. Alerts that are not yet classified get `budget_exhausted`.
- B8 **Failsafes.** `max_steps` and a no-progress detector, each ending the run with its own terminal status.
- B9 **report.json.** Validates against `report.schema.json` with one row per alert, and is still written when the run crashes.
- B10 **All three targets.** Runs end to end with the staff `[scan]` blocks.
-

Part C: evidence and README (3)

- C1 **README.** Follows the README contract in Section 10: checks R1 to R8 pass. This covers an end-to-end run on each target and one run that shows the budget cutoff.
- C2 **Results.** For each target, the counts of label 1, label 0 and null. On OWASP, precision and recall against the benchmark labels, and how you used them to refine the agent (accuracy is reported, not graded). One sequence diagram of one alert's trajectory, produced by the trajectory command of Section 8 from a committed trace.
- C3 **Limitations and alternatives.** A comparison with at least two other architectures (fixed workflow, plan-and-execute, supervisor/worker) on cost, latency, coordination and failure behaviour. Analyse them; do not build them.
-

Grading notes:

- Classification accuracy is not graded.
- A missing mechanism cannot earn its point.
- Graders run B7 with a small `max_tokens`.

10 README contract

A grading skill and the TAs read your `README.md`. It must follow this contract exactly, so that every submission can be graded the same way. Start from `README.template.md`, which has the required headings and their instructions.

Format rules

- The file is `README.md` at the repository root, in GitHub-flavoured Markdown.
- The level-2 headings below appear **in this order and with exactly this text**. You may add other headings only as level-3 headings inside a required section, except where a required level-3 heading is named below.
- Every command is in a fenced code block tagged `sh`, one command per line, with no prompts (\$) and no output mixed into the block.
- Commands run from the repository root, on macOS or Linux, in a fresh clone.
- Commands contain no placeholders (`<...>`, `YOUR_KEY`, ...). The only exception is secrets, which are read from environment variables named in `## Requirements`.
- No command is interactive. No command needs a secret written into a file in the repository.
- Paths are relative to the repository root.

Required sections

The headings, in order:

```
## Team
## Requirements
## Setup
## Fetch targets
## Configs
## Run
### Radicale
### SGLang
### OWASP BenchmarkPython
### Budget cutoff
## Outputs
## Validate
## Trajectory
## Rubric map
## Known issues
```

#	Heading	Must contain
1	## Team	Each member's name and UNI. The full commit SHA of the submitted code.
2	## Requirements	The operating system(s) tested. Language and runtime versions. The Python 3.12 and Bandit 1.9.4 install. The model provider and exact model name. A table of every environment variable the agent reads (name and purpose, never the value).
3	## Setup	One sh block that takes a fresh clone to a ready state: dependencies installed, Bandit 1.9.4 on the path. The last line prints the Bandit version (for example bandit --version).
4	## Fetch targets	One sh block that clones all three targets into targets/radicale , targets/sglang and targets/owasp-benchmark-python , checks out the pinned commits from Section 3, and prints git rev-parse HEAD for each.
5	## Configs	A table with columns Target , Config file . One row per target, plus one row for the cutoff run. Each config file is committed. Its [scan] block is identical to Section 4.
6	## Run	Exactly four level-3 subsections, in this order: ### Radicale , ### SGLang , ### OWASP BenchmarkPython , ### Budget cutoff . Each holds one sh block with one agent command in the Section 5 form, writing to runs/radicale , runs/sglang , runs/owasp-benchmark-python and runs/radicale-cutoff respectively. Under each block: the observed wall-clock time and used_tokens . The cutoff run uses the Radicale target with a config whose max_tokens is small enough that at least one row is budget_exhausted .
7	## Outputs	A table with columns Run , Directory , alert_count , label_1 , label_0 , null , used_tokens , exhausted . One row per run in ## Run . The values must equal the committed report.json in each directory.
8	## Validate	One sh block that validates every committed report.json against report.schema.json and exits non-zero if any fails.
9	## Trajectory	One sh block with one command that renders the trajectory of one alert from runs/radicale/trace.jsonl as a Mermaid sequenceDiagram (Section 8), then the rendered diagram in a fenced block tagged mermaid .
10	## Rubric map	A table with columns Item , Where . One row for each of the 20 rubric items (A1 to C3), in rubric order. Where is a path:line or path:start-end into the code for Part B items, and a section heading of the report for Part A and C items.
11	## Known issues	Anything that does not work, failed runs, and deviations from the handout. Write None if there are none.

Checks the grading skill applies

Each check passes or fails. The skill reports every failure with the check id and the line in **README.md**.

Id	Check
R1	All eleven level-2 headings are present, in order, with the exact text.
R2	## Run has the four level-3 subsections, in order, each with exactly one sh block containing one command.
R3	Every sh block follows the format rules: no placeholders, no \$ prompts, no interactive commands.
R4	Each run command uses --input , --output and --config , with the target directory, output directory and config file named in rows 4 to 6.
R5	Each config file listed in ## Configs exists, and its [scan] block equals Section 4 for that target.
R6	Each output directory in ## Outputs exists and contains report.json and trace.jsonl .
R7	The numbers in ## Outputs equal the counts computed from each committed report.json .
R8	The cutoff run's report.json has budget.exhausted = true and at least one budget_exhausted row.
R9	## Rubric map has 20 rows, A1 to C3 in order. Each path:line exists in the repository, and each report heading exists in the report.
R10	No file in the repository contains an API key (the skill scans for common key patterns).
R11	Each trace.jsonl in the four run directories meets the invariants in Section 8.

How the checks are used

- C1 is earned only if R1 to R8 all pass.
- R9 does not earn a point, but graders look for each rubric item first at the place the map points to. If a row is missing or wrong, the grader searches the submission and notes it.
- R11 does not earn a point by itself. Graders use it as evidence for B5.
- A failure of R10 is reported to the instructor. It is not a rubric deduction.
- The skill checks the README and the committed outputs. It does not run the agent. TAs may run the **## Setup**, **## Fetch targets** and **## Run** commands to confirm B10.

11 Deliverables

- Source code, and a lockfile or equivalent that pins dependencies.
- **README.md** following the README contract (Section 10).
- The four run directories named in the README (**runs/radicale**, **runs/sglang**, **runs/owasp-benchmark-python**, **runs/radicale-cutoff**) and their configs.
- The design report (Part A plus C2 and C3), as **REPORT.md** or a PDF.

- For each target, one unedited output directory (`report.json` and `trace.jsonl`) and the config used. No API keys in the repository.
- A contribution statement that says who did what.

12 Academic integrity and AI tools

AI assistants may be used to learn, debug your own code, review a design, or provide scoped autocomplete. Do not hand the assignment to an agent and submit the result. You remain responsible for every line you submit. Every team member must be able to explain the entire submission; the individual exams test this.

You may not use agents to generate or modify the target code. The guideline on LLM use is strictly enforced: use LLMs to learn, not to do the assignment for you.